

A Remediation Guide for the *RRRM-2 Kidney Transcriptome* Pipeline

Concrete Fixes, Code Sketches, and Validation Plans for the
Inferential and Phase 8 Issues Identified in the Audit

Automated Remediation Plan

Companion document to `critique_report.tex`

Audited run: `run_20260408_191759_2500g` (Git 90ee5bf)

April 26, 2026

Abstract

This document is a companion to the audit `critique_report.tex` and translates each identified weakness into a concrete remediation. Fourteen fixes are documented across two tiers. Tier 1 (Fixes 1–9) addresses the inferential layer: the broken Stouffer aggregation in Phase 6, the post-hoc focused-permutation FDR, the silent-shifter DE-attachment and null-calibration failure, the anchor-gene pre-registration mismatch, the documentation drift between `config/hyperparameters.yaml` and the executed code, the MuSiC reference-atlas sensitivity gap, the metric mismatch between Phase 3’s cosine-distance statistic and Phase 6’s edge-sum permutation null, and the absent external validation. The repository now implements the code/config/documentation side of these remediations; full numerical claims still require rerunning the affected phases on the remediated pipeline. Tier 2 (Fixes 10–14) addresses the structural cause of the Phase 8 negative result: at $n=5$ biological replicates per Age $\times$ Arm $\times$ EnvGroup cell, leave-one-out LIONESS weights inherit excessive sampling variance, and per-sample network features did not beat a simple expression baseline in the audited run. The five Tier 2 interventions (LIONESS pooling across cells, alternative sample-specific-network methods, alternative feature engineering, continuous targets, and multi-study cohort pooling for the LAR-Young cell with OSD-102) each target this noise floor directly. External replication and pooling use **OSD-102** (RR-1 NASA Validation Flight, female C57BL/6J kidney, $n=6+6$, 16 weeks, 37-day flight) as the primary partner and **OSD-513** (RR-23, male C57BL/6J kidney, $n=9+9+9$, 16–17 weeks, 38-day flight) as the sex-stratification secondary; OSD-568 is *not* compatible with OSD-771 (BALB/cAnNTac vs C57BL/6NTac) and is removed. For every fix the guide specifies the affected files, the historical behavior, the implemented replacement or guarded module, validation procedures, dependencies, and remaining limits.

Contents

1	How to Use This Guide	5
2	Triage Summary	5
	Tier 1 — Inferential Layer Fixes	7
3	Fix 1 — Replace Stouffer Aggregation with Signed Empirical Aggregation	7
3.1	Problem	7
3.2	Affected Files and Lines	7
3.3	Current Behavior	7

3.4	Proposed Behavior	8
3.4.1	Empirical Brown Implementation	8
3.4.2	Label-Permutation Implementation	9
3.5	Validation	9
3.6	Effort and Dependencies	10
4	Fix 2 — Hierarchical Benjamini–Bogomolov FDR	10
4.1	Problem	10
4.2	Affected Files and Lines	10
4.3	Current Behavior	10
4.4	Proposed Behavior	10
4.5	Validation	11
4.6	Effort and Dependencies	12
5	Fix 3 — Reconcile RESULTS.md with the Latest Run	12
5.1	Problem	12
5.2	Affected Files and Lines	12
5.3	Specific Edits Required	12
5.4	Validation	13
5.5	Effort and Dependencies	13
6	Fix 4 — External Validation on OSD-102 (Primary) and OSD-513 (Sex-Stratification)	13
6.1	Problem	13
6.2	Affected Files and Lines	14
6.3	Pre-Registration Protocol	14
6.4	Implementation Sketch	15
6.5	Validation	16
6.6	Effort and Dependencies	16
7	Fix 5 — Silent-Shifter Null Calibration	17
7.1	Problem	17
7.2	Affected Files and Lines	17
7.3	Implemented Route	17
7.4	Implementation Sketch	17
7.5	Validation	18
7.6	Effort and Dependencies	18
8	Fix 6 — Anchor Pre-Registration Enforcement	18
8.1	Problem	18
8.2	Affected Files and Lines	19
8.3	Implementation Sketch	19
8.4	Validation	19
8.5	Effort and Dependencies	20
9	Fix 7 — Config-Code Drift Removal	20
9.1	Problem	20
9.2	Affected Files and Lines	20
9.3	Implementation Sketch	20
9.4	Validation	21
9.5	Effort and Dependencies	22
10	Fix 8 — MuSiC Reference-Atlas Sensitivity Analysis	22

10.1 Problem	22
10.2 Affected Files and Lines	22
10.3 Implementation Sketch	22
10.4 Downstream Sensitivity Validation	23
10.5 Effort and Dependencies	23
11 Fix 9 — Permutation-Statistic Renaming and Appendix-Tier Full-Pipeline	
Permutation	23
11.1 Problem	23
11.2 Affected Files and Lines	23
11.3 Cheap Fix: Rename and Document	24
11.4 Expensive Fix: Full-Pipeline Permutation on Top Hits	24
11.5 Validation	25
11.6 Effort and Dependencies	25
Tier 2 — Phase 8 Design Interventions	26
12 Fix 10 — LIONESS Pooling Across Age×Arm×EnvGroup Cells	26
12.1 Problem	26
12.2 Pooling Strategy	26
12.3 Affected Files and Lines	26
12.4 Implementation Sketch	26
12.5 Validation	27
12.6 Effort and Dependencies	27
13 Fix 11 — Alternative Sample-Specific-Network Methods	27
13.1 Problem	27
13.2 Affected Files and Lines	28
13.3 Methods to Implement	28
13.4 Implementation Sketch	28
13.5 Validation	29
13.6 Effort and Dependencies	29
14 Fix 12 — Alternative Feature Engineering for Phase 8	29
14.1 Problem	29
14.2 Affected Files and Lines	29
14.3 Feature Sets to Compare	29
14.4 Implementation Sketch	29
14.5 Validation	30
14.6 Effort and Dependencies	30
15 Fix 13 — Continuous Classification Targets	30
15.1 Problem	30
15.2 Affected Files and Lines	30
15.3 Stress-Score Definition	31
15.4 Implementation Sketch	31
15.5 Validation	31
15.6 Effort and Dependencies	31
16 Fix 14 — Multi-Study Cohort Pooling for LAR-Young (OSD-102 + OSD-771)	32
16.1 Problem	32
16.2 Affected Files and Lines	32
16.3 Implementation Strategy	32

16.4 Implementation Sketch	33
16.5 Validation	34
16.6 Effort and Dependencies	34
17 Implementation Roadmap	35
17.1 Dependency Graph	35
17.2 Total Effort	35
17.3 Minimum-Viable Path to Publication	35
17.4 Risk Register	36
18 Acceptance Criteria	36

1 How to Use This Guide

This guide is structured for a single-engineer remediation campaign. Read §17 first for the dependency graph and recommended ordering. For each fix, the layout is:

- **Problem.** A one-paragraph statement of what goes wrong, sourced from the audit and grounded in the realized outputs of `run.20260408.191759.2500g`.
- **Affected files and lines.** The module(s) and approximate line range to edit.
- **Current behavior.** A code excerpt showing what the pipeline does today.
- **Proposed behavior.** A code sketch showing the replacement, written to be drop-in where possible.
- **Validation.** A test or sanity check that confirms the fix landed correctly.
- **Effort.** An honest estimate in person-days.
- **Dependencies.** Other fixes or external artifacts that must precede this one.

Code sketches are deliberately incomplete in places that depend on details of the existing module (variable names, helper functions, data-loader signatures). Treat them as scaffolds, not as drop-in replacements; the scaffold should let an engineer working with the existing repository land the fix in one sitting.

2 Triage Summary

#	Fix	Primary file	Effort	Tier
1	Signed empirical label-permutation gene aggregation	<code>src/statistics/full_regression.py</code>		1
2	Hierarchical Benjamini-Bogomolov FDR	<code>src/statistics/permutation_bootstrap.py</code>		
3	RESULTS.md reconciliation with latest run	RESULTS.md	1 h	1
4	External validation on OSD-102 (primary) and OSD-513 (sex-stratification)	<code>src/validation/external_replication.py</code> (new)		1
5	Silent-shifter null calibration	<code>src/statistics/silent_shifters.py</code>	0.5 d	1
6	Anchor pre-registration enforcement	<code>src/networks/procrustes.py</code>	1 h	1
7	Config-code drift removal	<code>config/hyperparameters.yaml</code> , <code>tests/</code> (new)	1 h	1
8	MuSiC reference-atlas sensitivity analysis	<code>src/preprocessing/deconvolution_sensitivity.R</code> (new)		
9	Permutation-statistic rename and full-pipeline appendix run	<code>src/statistics/permutation_bootstrap.py</code> , new module	cluster time	
10	LIONESS pooling across Age×Arm×EnvGroup cells	<code>src/networks/lioness.py</code> , <code>src/validation/cross_validation.py</code>	1 d	2
11	Alternative sample-specific-network methods (SSN, CSN, scLink)	<code>src/networks/alternative_methods.py</code> (new)	3 d	2
12	Alternative feature engineering for Phase 8	<code>src/validation/cross_validation.py</code>	1.5 d	2
13	Continuous classification targets (KIM-1, NGAL, HAVCR1)	<code>src/validation/continuous_target.py</code> (new)	2 d	2

#	Fix	Primary file	Effort	Tier
14	Multi-study cohort pooling for LAR-Young (OSD-102 + OSD-771)	src/validation/multi_study_pool.py (new)	4	2

Tier 1 fixes are necessary for publication. Tier 2 fixes are the only interventions that have a realistic chance of moving Phase 8 from a negative result to a positive one; they are scientific design changes, not analysis cleanups, and may be appropriate for a follow-up study rather than the current manuscript.

Tier 1 — Inferential Layer Fixes

3 Fix 1 — Replace Stouffer Aggregation with Signed Empirical Aggregation

3.1 Problem

`src/statistics/full_regression.py` fits a per-edge OLS model and combines per-edge two-tailed p -values into per-gene p -values via Stouffer's Z . Stouffer assumes the per-edge p -values are mutually independent. On a fixed shared-topology skeleton built from a partial-correlation graph, this assumption is violated: edges sharing a common endpoint are correlated through the precision-matrix structure, and edges within densely connected subgraphs (e.g., gene neighborhoods enriched in a pathway) are even more correlated. The realized outputs in `phase6_regression/` show the symptom directly: top genes have `p_stouffer = 0.0` numerically, despite per-edge median $|t| < 1$ for the same genes. With ~ 80 –440 edges incident per gene and per-edge $|t| < 1$, Stouffer's $Z = \sum_e t_e / \sqrt{n_e}$ should fall in the moderate-evidence range, not in the tail of the standard normal beyond machine epsilon. The saturation at $p = 0$ is anti-conservative inflation driven by edge-level dependence.

The numerical impact is large. The current run reports 154 genes at $q < 0.05$ for the main flight effect, 251 for the three-way interaction, 591 for the arm–flight interaction, and 25 for the age–flight interaction. None of these counts is interpretable as a frequentist FDR-controlled discovery rate.

3.2 Affected Files and Lines

- `src/statistics/full_regression.py` — `stouffer_combine` function and the per-gene aggregation loop.
- Optional: a new module `src/statistics/dependent_combine.py` that encapsulates Brown's method and Lancaster's method.

3.3 Current Behavior

```

1  def stouffer_combine(p_values, weights=None):
2      """Combine p-values via Stouffer's Z. Assumes independence."""
3      p = np.clip(p_values, 1e-300, 1.0)
4      z = norm.isf(p) # one-sided z for each edge
5      if weights is None:
6          z_combined = z.sum() / np.sqrt(len(z))
7      else:
8          z_combined = (weights * z).sum() / np.sqrt((weights**2).sum())
9      return float(norm.sf(z_combined))
10
11 # Per-gene loop (representative)
12 for gene in genes:
13     edges_g = edges_incident(gene) # ~80-440 edges
14     pvals_g = edge_pvals[edges_g] # from per-edge OLS
15     p_gene = stouffer_combine(pvals_g) # SATURATES TO 0
16     out.append((gene, p_gene))

```

Listing 1: Current Stouffer aggregation (representative excerpt).

3.4 Proposed Behavior

The implemented replacement is a signed incident-edge t statistic calibrated by within-Age $\times$ Arm label permutation. This fixes both failures at once: edge evidence keeps direction, and the gene-level null is empirical rather than assuming independent incident edges. Brown’s method remains an acceptable diagnostic or partial dependence correction, but it is not the primary inferential aggregate in the remediated repository. Two implementation paths are appropriate:

- **Empirical Brown.** Estimate the covariance of the per-edge t -statistics across the 80 samples directly from the residualized expression matrix; this requires no extra fits.
- **Label-permutation aggregation.** Generate K label-permutations within each Age $\times$ Arm stratum, recompute the per-gene aggregate t^2 statistic on each, and report the empirical p -value. This is the slower but exact alternative; it is also the same null already used in Phase 6’s rewiring permutation, ensuring methodological coherence.

3.4.1 Empirical Brown Implementation

```

1 import numpy as np
2 from scipy.stats import chi2
3
4 def empirical_brown_combine(p_values, edge_t_matrix):
5     """
6     Combine per-edge p-values via Brown’s empirical method.
7
8     Parameters
9     -----
10    p_values : (E,) array of per-edge p-values
11    edge_t_matrix : (E, N) array of per-edge t-statistics across N samples
12                  (used to estimate the empirical correlation among edges)
13
14    Returns
15    -----
16    p_brown : float, dependence-corrected combined p-value
17    """
18    p = np.clip(p_values, 1e-300, 1.0)
19    chi2_stat = -2.0 * np.log(p).sum() # Fisher chi^2 statistic
20
21    # Estimate correlation matrix of edge t-statistics (E x E)
22    # Use vectorized standardization for efficiency
23    T = edge_t_matrix - edge_t_matrix.mean(axis=1, keepdims=True)
24    T = T / (T.std(axis=1, keepdims=True) + 1e-12)
25    # Pairwise correlations: only upper triangle is needed
26    k = T.shape[0]
27    cov_pairs = (T @ T.T) / T.shape[1]
28    np.fill_diagonal(cov_pairs, 0.0)
29    sum_cov = cov_pairs[np.triu_indices(k, k=1)].sum()
30
31    # Brown (1975) variance correction:
32    # Var(chi2_stat) = 4k + 2 * sum_{i<j} f(rho_ij)
33    # Empirical Brown uses an empirical f(rho) = 3.263 * rho + 0.710 * rho^2
34    # (Poole et al. 2016) which is more accurate than Brown’s original
35    # formula.
36    rho = cov_pairs[np.triu_indices(k, k=1)]
37    f_rho = 3.263 * rho + 0.710 * rho**2
38    variance_correction = 4.0 * k + 2.0 * f_rho.sum()
39
40    # Rescale chi^2 to a chi^2(df) reference
41    expected = 2.0 * k
42    c = variance_correction / (2.0 * expected)
43    df = (2.0 * expected**2) / variance_correction
44    p_brown = chi2.sf(chi2_stat / c, df=df)

```



```
44     return float(p_brown)
```

Listing 2: Empirical Brown’s method for per-gene aggregation.

3.4.2 Label-Permutation Implementation

```
1  def label_perm_aggregate(z_edges, ii, jj, labels, K=5000, seed=0):
2      """
3      For each gene g, compute observed sum-|t| over edges incident on g,
4      then build a null by permuting labels within each stratum K times
5      and recomputing the same gene-level statistic.
6
7      Parameters
8      -----
9      z_edges : (N, E) LIONESS Fisher-z weights (samples x edges)
10     ii, jj : (E,) edge endpoint indices into the gene array
11     labels : (N,) condition labels (FLT vs GC), already restricted to a
12             stratum
13     K : permutation count
14
15     Returns
16     -----
17     p_perm : (G,) per-gene empirical p-value
18     """
19     rng = np.random.default_rng(seed)
20     G = max(ii.max(), jj.max()) + 1
21
22     def gene_aggregate(t_per_edge):
23         # Sum |t| over edges incident on each gene
24         out = np.zeros(G)
25         np.add.at(out, ii, np.abs(t_per_edge))
26         np.add.at(out, jj, np.abs(t_per_edge))
27         return out
28
29     # Observed per-edge t (two-sample t between labels)
30     t_obs = welch_t(z_edges, labels) # (E,)
31     obs_gene = gene_aggregate(t_obs) # (G,)
32
33     # Null distribution
34     counter = np.zeros(G, dtype=np.int64)
35     for k in range(K):
36         perm_labels = rng.permutation(labels)
37         t_k = welch_t(z_edges, perm_labels)
38         null_gene = gene_aggregate(t_k)
39         counter += (null_gene >= obs_gene)
40     p_perm = (counter + 1) / (K + 1)
41     return p_perm
```

Listing 3: Label-permutation aggregation, vectorized over genes.

3.5 Validation

Three sanity checks should be added as unit tests in `tests/test_dependent_combine.py`:

1. **Identity check.** If the input edge p -values are mutually independent (e.g., generated as $p = \text{np.random.uniform}(0, 1, \text{size}=k)$ from a uniform null), the empirical-Brown p -value should match Stouffer’s p -value to within Monte Carlo error. If it doesn’t, the empirical correction is over-correcting.
2. **Saturation check.** For a gene with per-edge median $|t| < 1$, the combined p -value must

satisfy $p_{\text{brown}} > 10^{-50}$. If it doesn't, the dependence correction is not sufficient and label-permutation should be used.

3. **Calibration check.** On a gene with simulated null edge t-statistics drawn from a multivariate normal with a known correlation matrix, the empirical-Brown p -value should be uniform on $[0, 1]$ across ≥ 1000 Monte Carlo replications (Kolmogorov–Smirnov test, $p > 0.05$).

3.6 Effort and Dependencies

Effort: 0.5 person-days for empirical-Brown plus tests; 1 person-day if the label-permutation alternative is also implemented. **Dependencies:** none. The fix lands cleanly inside the existing module.

4 Fix 2 — Hierarchical Benjamini–Bogomolov FDR

4.1 Problem

`src/statistics/permutation_bootstrap.py` implements a “focused permutation” option that restricts the BH correction domain to the top decile of genes ranked by observed rewiring magnitude before applying BH. The repository defends this as “the top-decile cutoff is defined by rank order of the observed rewiring, not by the permutation p -values themselves, avoiding circular inference.” This defense is incorrect: under any reasonable null, observed rewiring magnitude and permutation p -value are monotonically related, so restricting BH to the top decile is post-hoc selection on a quantile of the test statistic. The result is anti-conservative.

4.2 Affected Files and Lines

- `src/statistics/permutation_bootstrap.py` — the `--focused-permutation` branch and its BH application.
- Optionally: a new helper module `src/statistics/hierarchical_fdr.py`.

4.3 Current Behavior

```

1  if args.focused_permutation:
2      # Define top-decile by OBSERVED rewiring magnitude
3      cutoff = np.quantile(rewiring_obs, args.focused_quantile) # e.g. 0.90
4      in_domain = rewiring_obs >= cutoff
5      # Apply BH only on the in-domain subset
6      p_in = p_perm[in_domain]
7      q_in = bh_fdr(p_in)
8      q_full = np.full_like(p_perm, np.nan)
9      q_full[in_domain] = q_in

```

Listing 4: Current focused-permutation BH (representative excerpt).

4.4 Proposed Behavior

The Benjamini–Bogomolov (2014) hierarchical procedure is the cleanest formal replacement. The procedure is:

1. **Stage 1.** Test all M pathways for rewiring enrichment. Apply BH at level α to the pathway p -values. Let $\mathcal{P}^*$ be the set of pathways significant at stage 1.
2. **Stage 2.** For each pathway $P \in \mathcal{P}^*$, apply BH at level $\alpha \cdot |\mathcal{P}^*|/M$ to the gene-level p -values within P .

For pre-specified non-overlapping families, the level adjustment gives the cleanest FDR interpretation. When gene sets overlap, the remediated code reports conservative BY sensitivity columns and the limitation must be stated explicitly. The procedure recovers the power advantage that focused permutation was trying to claim — i.e., it concentrates testing on a smaller, biologically plausible domain — without selecting the testing set from the observed top-decile statistic.

```

1  def benjamini_bogomolov(pathway_pvals, gene_pvals_per_pathway, alpha=0.05):
2      """
3      Two-stage hierarchical FDR (Benjamini & Bogomolov, JRSS-B 2014).
4
5      Parameters
6      -----
7      pathway_pvals : dict {pathway_name: p_value}
8      gene_pvals_per_pathway : dict {pathway_name: dict {gene: p_value}}
9      alpha : nominal FDR level
10
11     Returns
12     -----
13     significant_pathways : list of pathways significant at stage 1
14     significant_genes : dict {pathway: list of significant genes at stage 2}
15     """
16     M = len(pathway_pvals)
17     # Stage 1: BH on pathway p-values
18     pwy = list(pathway_pvals.keys())
19     pwy_p = np.array([pathway_pvals[p] for p in pwy])
20     pwy_q = bh_fdr(pwy_p)
21     sig_pathways = [pwy[i] for i in range(M) if pwy_q[i] < alpha]
22     M_star = len(sig_pathways)
23     if M_star == 0:
24         return [], {}
25
26     # Stage 2: BH on gene p-values, level alpha * M_star / M
27     sig_genes = {}
28     level2 = alpha * M_star / M
29     for P in sig_pathways:
30         gene_p = gene_pvals_per_pathway[P]
31         names = list(gene_p.keys())
32         pvals = np.array([gene_p[g] for g in names])
33         qvals = bh_fdr(pvals)
34         sig_genes[P] = [names[i] for i in range(len(names)) if qvals[i] <
35                             level2]
36
37     return sig_pathways, sig_genes

```

Listing 5: Hierarchical Benjamini–Bogomolov implementation.

4.5 Validation

Three checks:

1. **Null simulation.** Generate $K = 1000$ datasets under a known null where no pathway is enriched and no gene is differentially rewired. For non-overlapping pre-specified families, the Benjamini–Bogomolov procedure should remain near the nominal level; for overlapping families, the implementation must report the conservative BY-adjusted sensitivity columns and document that strict independence guarantees do not apply.
2. **Power simulation.** Generate $K = 1000$ datasets where one pre-specified pathway is genuinely enriched and 5 of its 50 member genes are rewired. The hierarchical procedure should detect both stages with power ≥ 0.5 at the OSD-771 sample size; if not, the level adjustment is too aggressive and a less stringent pre-registered candidate-only BH analysis should be

reported separately.

3. **Empirical comparison.** On the realized OSD-771 outputs, compare full-domain BH, pre-registered candidate-only BH, and hierarchical FDR outputs. Do not run the old observed-top-decile focused permutation as an inferential comparator; it is retained only as an audit example of invalid selective testing.

4.6 Effort and Dependencies

Effort: 1 person-day. **Dependencies:** the pathway-level p -values from the existing pathway permutation variant must be available; if not, the pathway permutation must be run first.

5 Fix 3 — Reconcile RESULTS.md with the Latest Run

5.1 Problem

The current RESULTS.md cites gene-level findings (11 ISS-T-Young hits at $FDR < 0.05$ under focused permutation; *Slc6a20a*, *Trf*, *Cry2*, *Gc*, *Pfkfb*, *Foxq1*; retinol-metabolism OR= 19.9, $q = 1.6 \times 10^{-5}$ in LAR Young) and a $B=50$ bootstrap that are not consistent with run_20260408_191759_2500g. The latest run uses $B=2000$ and $K=5000$, yields 24 ISS-T-Young hits at $FDR < 0.05$ under full-domain BH (not 11), and shows translation/ribosome/NMD pathways and PPAR signaling as the dominant cross-contrast biology rather than retinol metabolism.

5.2 Affected Files and Lines

- RESULTS.md — entire file, but especially the headline-findings section, the gene-list section, the pathway-enrichment section, and any reference to bootstrap iteration counts.
- METHODOLOGY.md — the Phase 6 section.
- walkthrough.md — any summary numbers.

5.3 Specific Edits Required

- Add a “Run identifier and provenance” section at the top of RESULTS.md: “All numerical claims in this document are derived from data/results/run_20260408_191759_2500g/ (Git commit 90ee5bf, 2026-04-08). Earlier runs cited in prior versions of this document have been superseded.”
- Replace any reference to “ $B=50$ ” with “ $B=2000$ ” globally.
- Replace “11 genes at $FDR < 0.05$ in ISS-T Young” with “24 genes at $FDR < 0.05$ in ISS-T Young under full-domain BH; 0 in ISS-T Old, 0 in LAR Young, and 0 in LAR Old at $q < 0.05$ under full-domain BH; 6, 6, and 0 genes respectively at $q < 0.10$.”
- Replace the retinol-metabolism headline with the translation-machinery and PPAR signaling cluster: “ISS-T Old shows convergent enrichment of translation-machinery pathways (ribosome, eukaryotic translation elongation, peptide chain elongation, selenocysteine synthesis, NMD-Enhanced, NMD-Independent, viral mRNA translation, GCN2-amino-acid-deficiency response) at $q \approx 0.005$ across KEGG and Reactome libraries. LAR Young is dominated by PPAR signaling at $q = 1.77 \times 10^{-3}$ (KEGG) and the same translation-machinery cluster at $q \approx 0.03$. LAR Old shows both signatures plus activated NOTCH1 signaling. ISS-T Young, despite the largest gene-level hit count, has the weakest pathway-level signal (best $q \approx 0.28$).”
- Add a “Phase 8b: Validation Negative Result” section that displays the enhanced-CV table directly, states that the audited run’s feature construction was not fully fold-safe, and explains

why the negative conclusion remains credible because network features lost to the expression baseline even with that favorable leakage risk. The remediated rerun must recompute LIONESS/features/scaling/PCA/feature selection inside each fold.

- Add a “Caveats and Limitations” section enumerating the post-hoc focused-permutation issue (resolved by candidate-only or hierarchical FDR per Fix 2), the Stouffer aggregation issue (resolved by signed empirical label-permutation per Fix 1), the silent-shifter DE-attachment issue (per Fix 5), and the absence of completed external cohort replication (guarded by Fix 4 once OSD-102 and OSD-513 are run).

5.4 Validation

RESULTS.md should be searchable for “ $B=50$ ” and “retinol metabolism” with zero hits after the edit, and for “ $B=2000$ ” and “translation-machinery” with at least one hit each. The Run identifier and provenance section should match `run_metadata.json` exactly.

5.5 Effort and Dependencies

Effort: 1 hour for mechanical edits; an additional 1–2 hours if the biological narrative needs to be rewritten substantially. **Dependencies:** Fixes 1 and 3 should ideally land first so that the new RESULTS.md reflects the post-fix gene counts rather than the pre-fix counts.

6 Fix 4 — External Validation on OSD-102 (Primary) and OSD-513 (Sex-Stratification)

6.1 Problem

The current pipeline trains and tests every inferential layer on a single cohort (OSD-771). Even after Fix 2 legitimizes the gene-level FDR claims, the survival of those claims in an independent cohort is the single most decisive piece of evidence a peer reviewer can ask for.

The originally proposed candidate (OSD-568, RRRM-1 kidney) is *not* compatible with OSD-771 and has been removed from this guide. RRRM-1 used female BALB/cAnNTac mice while RRRM-2 uses female C57BL/6NTac — BALB/c and C57BL/6 differ in baseline kidney transcriptome by an order of magnitude more than typical batch effects, and OSD-568 itself appears (from the OSDR release announcement) to be a miRNA-seq or scRNA-seq dataset rather than bulk RNA-seq. Pooling or replicating across BALB/c and C57BL/6 is not statistically defensible at the gene level.

The two viable replication cohorts on NASA GeneLab are OSD-102 (RR-1 NASA Validation Flight, female C57BL/6J kidney, $n=6$ FLT + $n=6$ GC, 16 weeks, 37-day flight, bulk RNA-seq) and OSD-513 (RR-23, male C57BL/6J kidney, $n=9$ FLT + $n=9$ HGC + $n=9$ VGC, 16–17 weeks, 38-day flight, bulk RNA-seq). OSD-102 maps cleanly onto the RRRM-2 LAR-Young Flight-vs-GC contrast and is the primary external validation cohort: same sex, same age, same hardware (Rodent Habitat), same diet (NuRFB), same preservation (RNAlater $\rightarrow -80^\circ\text{C}$), same light cycle (12h/12h), with only the C57BL/6J vs C57BL/6NTac sub-strain difference (the *Nnt* deletion in 6J is the single kidney-relevant divergence). OSD-513 maps onto the same contrast but with a sex mismatch and is used for sex-stratification: it asks which OSD-102-replicated findings survive in male mice. Together they constitute a tiered replication — primary in matched-sex, secondary in opposite-sex, with the differential reporting which findings are female-specific versus generally robust.

Independent replication does *not* require ComBat-seq. Each external cohort is analyzed independently and compared to the pre-registered RRRM-2 hypothesis set. ComBat-seq and PCA/batch

checks belong only to Fix 14, the separate OSD-102 plus OSD-771 LAR-Young pooling path.

The 24 ISS-T-Young hits cannot be validated in either cohort because neither has an ISS-T arm. They are reported as candidates pending an as-yet-unavailable ISS-T cohort.

6.2 Affected Files and Lines

- New module: `src/validation/external_replication.py`.
- Pre-registration directory: `docs/external_replication_protocol/`.
- Output directory: `data/results/external_replication/`.

6.3 Pre-Registration Protocol

The pre-registration protocol must be committed *before* any external data is loaded. This is the single most important step. The protocol fixes:

1. **Hypothesis set.** The post-Fix-2 hierarchical-BH-corrected hits from the LAR-Young Flight-vs-GC contrast (the only RRRM-2 contrast that maps onto OSD-102 and OSD-513), plus the translation-machinery and PPAR signaling pathway sets. The 24 ISS-T-Young hits are excluded because neither external cohort has an ISS-T arm.
2. **Replication criteria for OSD-102 (primary).** Per-gene rewiring direction agreement with RRRM-2 LAR-Young, per-gene $q < 0.10$ in OSD-102, pathway $q < 0.10$ in OSD-102.
3. **Replication criteria for OSD-513 (sex-stratification).** The OSD-102-replicated hits are pre-screened against a kidney sex-DE list (Ranjit et al. 2024 *Nature Genetics*; Tabula Muris Senis kidney sex-stratified atlas). Genes flagged as kidney-sex-DE are reported separately as “female-specific.” The remaining sex-independent subset is tested in OSD-513 with the same direction-and- q criteria.
4. **Pipeline parameters.** Identical to OSD-771 ($k=80$, $K=5000$, $B=2000$, 10 node2vec seeds, $d=128$, $p=0.25$, $q=4.0$).
5. **Sub-strain handling.** The C57BL/6J vs C57BL/6NTac sub-strain difference is treated as a covariate-of-no-interest in both cohorts. The *Nnt* deletion in 6J (which affects mitochondrial NADPH supply and may modulate oxidative-stress responses) is documented as a known confound.
6. **Recovery-window handling.** OSD-102 mice were sacrificed shortly post-splashdown; RRRM-2 LAR mice had 24 days of recovery before sacrifice. Any RRRM-2 hit that is specifically a *recovery-phase* phenomenon is expected to be absent from OSD-102, while any *persistent* signal is expected to be present (possibly stronger). The pre-registration declares this asymmetric expectation up front so that selective non-replication is not retrofitted as confirmation.
7. **Pool-sample exclusion.** OSD-102 contains technical pool samples (“Pool of all FLT samples,” “Pool of all GC samples,” “Pool of all FLT and GC samples,” “Empty Pool”); these are excluded from the per-mouse analysis. Use only individual mouse samples M23–M28 (FLT) and M33–M38 (GC), $n=12$ total.
8. **Interpretation rules.** (i) Surviving in OSD-102 only → “female-specific spaceflight signal.” (ii) Surviving in both OSD-102 and OSD-513 (after sex-DE filter) → “sex-robust spaceflight signal.” (iii) Surviving in OSD-513 only after sex-DE filter → “not female-specific” (informative but unusual — worth investigating). (iv) Failing both → “not externally validated; the original LAR-Young claim is retracted at the gene level.”

6.4 Implementation Sketch

```

1  # src/validation/external_replication.py
2  import argparse
3  from pathlib import Path
4  import numpy as np
5  import pandas as pd
6  from src.run_all_phases import run_phases  # existing orchestrator
7
8  LAR_YOUNG_CONTRAST = "LAR_YNG_FLT_minus_GC"
9
10
11 def run_external_replication(cohort, counts, meta, hypothesis_set,
12                             out_dir, sex_de_list=None):
13     """Run Phases 0-3 on external cohort, then test pre-registered hits."""
14     out = Path(out_dir) / cohort
15     out.mkdir(parents=True, exist_ok=True)
16
17     # Phase 0-3: deconvolution, residualization, network construction,
18     # embeddings. The hypothesis set is FIXED; we do not refit it here.
19     run_phases(phases=[0, 1, 2, 3], counts=counts, meta=meta,
20               output_dir=str(out))
21
22     hyp = pd.read_csv(hypothesis_set, sep='\t')
23     ext_rewiring = pd.read_csv(
24         out / f"phase3_rewiring/{LAR_YOUNG_CONTRAST}_rewiring_agg.tsv",
25         sep='\t'
26     )
27     ext_q = pd.read_csv(
28         out / f"phase6_uncertainty/{LAR_YOUNG_CONTRAST}_perm_pvals.tsv",
29         sep='\t'
30     )
31
32     merged = hyp.merge(ext_rewiring, on="gene", how="left",
33                       suffixes=('_osd771', f'_{cohort}'))
34     merged = merged.merge(ext_q[["gene", "q_BH"]], on="gene", how="left",
35                          suffixes=('', '_external'))
36
37     merged["direction_agree"] = (
38         np.sign(merged["rewiring_osd771"]) ==
39         np.sign(merged[f"rewiring_{cohort}"]))
40
41     merged["replicated"] = (
42         merged["direction_agree"] & (merged["q_BH_external"] < 0.10)
43     )
44
45     if sex_de_list is not None:
46         sex_de = set(pd.read_csv(sex_de_list, sep='\t')["gene"])
47         merged["is_kidney_sex_de"] = merged["gene"].isin(sex_de)
48     else:
49         merged["is_kidney_sex_de"] = False
50
51     merged.to_csv(out / "replication_report.tsv", sep='\t', index=False)
52     return merged
53
54
55 def main():
56     ap = argparse.ArgumentParser()
57     ap.add_argument("--cohort", required=True, choices=["OSD-102",
58               "OSD-513"])
59     ap.add_argument("--counts", required=True)
60     ap.add_argument("--meta", required=True)
61     ap.add_argument("--hypothesis-set",
62                     default="docs/external_replication_protocol/lar_young_hypothesis_set.ts

```

```

62     ap.add_argument("--sex-de-list",
63                     default="docs/external_replication_protocol/kidney_sex_de_genes.tsv",
64                     help="Used only for OSD-513 (sex stratification)")
65     ap.add_argument("--out", default="data/results/external_replication/")
66     args = ap.parse_args()
67
68     sex_de = args.sex_de_list if args.cohort == "OSD-513" else None
69     merged = run_external_replication(
70         args.cohort, args.counts, args.meta,
71         args.hypothesis_set, args.out, sex_de_list=sex_de,
72     )
73
74     n_total = len(merged)
75     n_repl = int(merged["replicated"].sum())
76     if args.cohort == "OSD-513":
77         non_sex_de = merged[~merged["is_kidney_sex_de"]]
78         n_sex_robust = int(non_sex_de["replicated"].sum())
79         print(f"[{args.cohort}] Replicated overall: {n_repl}/{n_total}")
80         print(f"[{args.cohort}] Sex-robust subset: "
81               f"{n_sex_robust}/{len(non_sex_de)}")
82     else:
83         print(f"[{args.cohort}] Replicated: {n_repl}/{n_total} at q<0.10")
84
85
86 if __name__ == "__main__":
87     main()

```

Listing 6: External-replication driver for OSD-102 (primary) and OSD-513 (sex-stratification).

6.5 Validation

Two replication reports are produced: `replication_report.tsv` for OSD-102 and a second for OSD-513. The headline numbers for the manuscript are (i) the OSD-102 replication rate among LAR-Young pre-registered hits, (ii) the OSD-513 replication rate within the sex-DE-filtered subset, and (iii) the cross-cohort consistency rate (genes that replicate in both). A replication rate ≥ 0.5 in OSD-102 is strong evidence for the LAR-Young claim; a sex-robust rate ≥ 0.5 in OSD-513 is strong evidence that the signal generalizes beyond female mice. A rate of 0 in either tells you the corresponding claim should be retracted. Either outcome is publishable.

A sanity check is mandatory before any replication claim. OSD-102 has been analyzed by multiple groups (Beheshti et al. spaceflight meta-analyses; Mishra et al. 2024 kidney work; the GeneLab consensus pipeline outputs). If the pipeline's per-gene effect estimates on OSD-102 are wildly inconsistent with these published RR-1 kidney effect estimates, the discrepancy must be resolved before any replication claim is made. This is not a circularity (the published estimates are not the hypothesis being tested) — it is a confirmation that the pipeline produces reasonable numbers on a well-characterized dataset.

6.6 Effort and Dependencies

Effort: 2 person-days for the script and pre-registration protocol; an additional 0.5–1 day for OSD-102 and OSD-513 data ingestion and metadata harmonization to match the OSD-771 meta-data schema. **Dependencies:** OSD-102 and OSD-513 must be downloadable from GeneLab; the kidney sex-DE list (Ranjit et al. 2024 *Nature Genetics* supplementary tables, or the equivalent Tabula Muris Senis kidney sex-stratified analysis) must be obtained and committed to `docs/external_replication_protocol/`; the pre-registration document must be committed and tagged in Git before any external data is loaded.

7 Fix 5 — Silent-Shifter Null Calibration

7.1 Problem

The silent-shifter construct is the repository’s signature biological category: genes with high rewiring (top decile) but low DE ($|\log_2 \text{FC}| < 0.3$, DE FDR > 0.2). The audited strict files, however, contained 250 rows per contrast because contrast-matched differential expression statistics were not actually attached. The reported 55–68 values were DE-supported counts within those rewiring-only files, not strict silent-shifter counts. This made the old tables internally inconsistent: they mixed candidate rewired genes, DE-supported rewired genes, and strict DE-null silent shifters without a shared null calibration.

7.2 Affected Files and Lines

- `src/statistics/silent_shifters.py` — entire module.

7.3 Implemented Route

Route A (formalize the existing construct). Attach the contrast-matched gene-level DE table before classification, then write separate tables for candidate rewired genes, DE-supported rewired genes, strict DE-null silent shifters, and the supported strict subset. Report observed-versus-expected calibration under independence for every contrast. If observed $<$ expected, the construct should be reframed as an enumeration of high-rewiring DE-null genes, not a positive discovery category.

Route B (optional future sensitivity). A residualized-rewiring statistic can still be used as an exploratory sensitivity analysis: regress per-gene rewiring on per-gene $|\log_2 \text{FC}|$, take the residuals, and define silent shifters as the top decile of *residualized* rewiring. This is not the primary remediated definition because the repository’s published construct is threshold-based.

7.4 Implementation Sketch

```

1 import numpy as np
2 import pandas as pd
3 from scipy.stats import binomtest
4
5 def silent_shifter_null_expectation(rewiring, lfc, fdr,
6                                     top_quantile=0.90,
7                                     lfc_max=0.3, fdr_min=0.2):
8     """Route A: report observed-vs-expected silent shifters under
9         independence."""
10    n_genes = len(rewiring)
11    cutoff = np.quantile(rewiring, top_quantile)
12    high_rewiring = rewiring >= cutoff
13    low_de = (np.abs(lfc) < lfc_max) & (fdr > fdr_min)
14    # Expected under independence
15    p_high = high_rewiring.mean()
16    p_lowde = low_de.mean()
17    expected = p_high * p_lowde * n_genes
18    observed = (high_rewiring & low_de).sum()
19    # One-sided binomial/hypergeometric-style calibration: enrichment over
20    null
21    test = binomtest(observed, n_genes, p_high * p_lowde,
22                    alternative='greater')
23    return {
24        "observed": int(observed),
25        "expected": float(expected),
26        "p_enrichment": float(test.pvalue),
27    }

```

```

24         "ratio": float(observed / max(expected, 1)),
25     }
26
27
28 def residualized_rewiring(rewiring, lfc):
29     """Route B: rewiring residualized on |log2FC|."""
30     from sklearn.linear_model import LinearRegression
31     X = np.abs(lfc).reshape(-1, 1)
32     y = np.asarray(rewiring)
33     model = LinearRegression().fit(X, y)
34     residuals = y - model.predict(X)
35     return residuals
36
37
38 def silent_shifters_residualized(rewiring, lfc, fdr,
39                                 top_quantile=0.90,
40                                 lfc_max=0.3, fdr_min=0.2):
41     """Route B: top decile of residualized rewiring with low DE."""
42     resid = residualized_rewiring(rewiring, lfc)
43     cutoff = np.quantile(resid, top_quantile)
44     high_resid = resid >= cutoff
45     low_de = (np.abs(lfc) < lfc_max) & (fdr > fdr_min)
46     return high_resid & low_de

```

Listing 7: Silent-shifter null calibration (Route A) and residualization (Route B).

7.5 Validation

For Route A, the observed-versus-null calibration should be reported alongside every silent-shifter table in `RESULTS.md`. Missing DE rows should fail the run by default. The strict table should no longer default to 250 genes per contrast unless those 250 genes genuinely satisfy the high-rewiring and DE-null criteria.

For Route B, validate by simulating: generate a dataset with known rewiring–DE correlation $\rho \in \{0, 0.1, 0.3, 0.5\}$ and check that the residualized procedure produces a silent-shifter set whose own $|\log_2 \text{FC}|$ distribution is centered at zero regardless of ρ .

7.6 Effort and Dependencies

Effort: 0.5 person-days. **Dependencies:** the per-gene DE statistics must be computed (already part of Phase 6).

8 Fix 6 — Anchor Pre-Registration Enforcement

8.1 Problem

`config/anchor_genes.yaml` lists pre-registered housekeeping, ribosomal, and core-metabolism genes intended to serve as Procrustes anchors — genes whose embeddings should be approximately invariant across contrasts so that orthogonal rotation can be estimated reliably. `src/networks/procrustes` actually selects anchors data-drivenly by reading Phase 3 rewiring tables and picking the bottom- K genes by median absolute rewiring across contrasts. This is the genes that already have low rewiring under the pipeline being chosen as the reference for measuring rewiring — a tautology.

The realized run uses 154 anchors (from `phase3_rewiring/anchors.txt`), all selected by this data-driven procedure. The list in `config/anchor_genes.yaml` is unused.

8.2 Affected Files and Lines

- `src/networks/procrustes.py` — the anchor-selection function.
- `config/anchor_genes.yaml` — already correct, but should be expanded to ensure ≥ 50 anchors after gene-panel intersection.

8.3 Implementation Sketch

```

1  import yaml
2  from pathlib import Path
3
4  def load_pre_registered_anchors(yaml_path, gene_panel):
5      """Load anchors from config/anchor_genes.yaml and intersect with
6         panel."""
7      with open(yaml_path) as f:
8          anchor_lists = yaml.safe_load(f)
9          # anchor_lists is a dict like {housekeeping: [...], ribosomal: [...],
10             ...}
11      all_anchors = set()
12      for category, genes in anchor_lists.items():
13          all_anchors.update(genes)
14      anchors_in_panel = sorted(all_anchors & set(gene_panel))
15      if len(anchors_in_panel) < 50:
16          raise ValueError(
17              f"Only {len(anchors_in_panel)} pre-registered anchors survive "
18              f"intersection with the gene panel of {len(gene_panel)}. "
19              f"Expand anchor_genes.yaml or relax the panel filter."
20          )
21      return anchors_in_panel
22
23  def select_anchors(gene_panel, mode="pre_registered",
24                    anchor_yaml="config/anchor_genes.yaml",
25                    rewiring_table=None, k=150):
26      """Anchor selection with explicit mode."""
27      if mode == "pre_registered":
28          return load_pre_registered_anchors(anchor_yaml, gene_panel)
29      elif mode == "data_driven":
30          # Existing behavior: bottom-k by median absolute rewiring.
31          # Should only be used for sensitivity analysis, not for headline
32          runs.
33          if rewiring_table is None:
34              raise ValueError("data_driven anchor mode requires
35                 rewiring_table")
36          median_abs_rewiring =
37              rewiring_table.groupby("gene")["abs_rewiring"].median()
38          return median_abs_rewiring.nsmallest(k).index.tolist()
39      else:
40          raise ValueError(f"Unknown anchor mode: {mode}")

```

Listing 8: Anchor selection with `-anchor-mode` flag.

8.4 Validation

A unit test should assert that, when `anchor_genes.yaml` is loaded and intersected with the 2500-gene panel of `run_20260408.191759_2500g`, at least 50 anchors survive. Procrustes alignment with the pre-registered anchor set should produce per-contrast rewiring distributions that are qualitatively similar to the data-driven selection but slightly more conservative (lower mean rewiring) because the pre-registered anchors are not pre-selected for low rewiring.

A sensitivity analysis should compare:

- Per-gene rewiring scores under `pre_registered` vs. `data_driven` anchor selection.
- Top-decile gene-list overlap between the two.
- FDR-significant counts under both selections.

If the lists diverge substantially, the gene-level claims are anchor-sensitive and should be reported as such.

8.5 Effort and Dependencies

Effort: 1 hour. **Dependencies:** `config/anchor_genes.yaml` must contain ≥ 50 panel-intersecting genes. If not, the YAML must be expanded first (an additional hour to curate housekeeping / ribosomal lists from MGI).

9 Fix 7 — Config-Code Drift Removal

9.1 Problem

Before remediation, `config/hyperparameters.yaml` specified $k = 50$ neighbors while `src/networks/shared_topo.py` used $k = 80$. The YAML specified a Random Forest with `n_estimators=100` and unbounded depth while `src/validation/cross_validation.py` used `max_depth=5`. The YAML also mentioned `westfall_young: true` even though no module implemented it. The remediated repository aligns config and execution at top-k 80, RF max depth 5, and implemented FDR modes only, with a test guard to prevent recurrence.

9.2 Affected Files and Lines

- `config/hyperparameters.yaml`.
- New: `tests/test_config_consistency.py`.
- Optional: a CI hook in `.github/workflows/` (or equivalent) that runs the config-consistency test.

9.3 Implementation Sketch

```

1 # config/hyperparameters.yaml
2 # Last verified consistent with src/ on 2026-04-26 against
3 # run_20260408_191759_2500g (Git 90ee5bf).
4
5 network_topology:
6   topk: 80                                # was 50, now matches shared_topology.py
7   shrinkage: ledoit_wolf
8   partial_correlation: true
9
10 node2vec:
11   dimensions: 128
12   p: 0.25
13   q: 4.0
14   walk_length: 80
15   num_walks: 200
16   window_size: 10
17   num_seeds: 10
18
19 permutation:
20   K: 5000

```

```

21 B_bootstrap: 2000
22 hierarchical_fdr:          # was: westfall_young, which is unimplemented
23     alpha: 0.05
24     method: benjamini_bogomolov
25
26 classifier:
27     random_forest:
28         n_estimators: 100
29         max_depth: 5          # was: null/unbounded, now matches
30         cross_validation.py
31         min_samples_leaf: 2
32     logistic_regression:
33         C: 1.0
34         solver: liblinear

```

Listing 9: Corrected hyperparameters.yaml (excerpt).

```

1 # tests/test_config_consistency.py
2 import yaml
3 from pathlib import Path
4
5 CONFIG_PATH = Path(__file__).resolve().parents[1] /
6     "config/hyperparameters.yaml"
7
8 def load_config():
9     with open(CONFIG_PATH) as f:
10         return yaml.safe_load(f)
11
12 def test_topk_matches_code():
13     config = load_config()
14     from src.networks.shared_topology import DEFAULT_TOPK # to be exported
15     assert config["network_topology"]["topk"] == DEFAULT_TOPK, (
16         f"YAML topk={config['network_topology']['topk']} but "
17         f"shared_topology.DEFAULT_TOPK={DEFAULT_TOPK}"
18     )
19
20 def test_rf_max_depth_matches_code():
21     config = load_config()
22     from src.validation.cross_validation import DEFAULT_RF_MAX_DEPTH
23     assert config["classifier"]["random_forest"]["max_depth"] ==
24         DEFAULT_RF_MAX_DEPTH
25
26 def test_no_unimplemented_methods():
27     config = load_config()
28     method = config["permutation"]["hierarchical_fdr"]["method"]
29     valid = {"benjamini_hochberg", "benjamini_bogomolov", "westfall_young"}
30     assert method in valid, f"Unknown FDR method: {method}"
31     if method == "westfall_young":
32         from src.statistics import permutation_bootstrap as pb
33         assert hasattr(pb, "westfall_young_correction"), (
34             "westfall_young method specified in YAML but not implemented in
35             code."
36         )

```

Listing 10: Config-consistency unit test.

9.4 Validation

`pytest tests/test_config_consistency.py` must pass. The test should be added to CI (GitHub Actions, Jenkins, or whatever the team uses) so that future drift is caught at PR time rather than at audit time.

9.5 Effort and Dependencies

Effort: 1 hour. **Dependencies:** the modules referenced by the test must export the relevant constants (DEFAULT_TOPK, DEFAULT_RF_MAX_DEPTH); these are 1-line additions.

10 Fix 8 — MuSiC Reference-Atlas Sensitivity Analysis

10.1 Problem

MuSiC deconvolution uses Tabula Muris Senis (TMS) as the single-cell reference. TMS samples male and female C57BL/6JN mice across six age groups (1, 3, 18, 21, 24, 30 months). OSD-771 uses female C57BL/6NTac mice at 16 and 34 weeks (approximately 4 and 8 months). The strain mismatch (C57BL/6JN vs. C57BL/6NTac) and the age-distribution mismatch are not addressed in the current pipeline, and any deconvolution bias propagates permanently into the global residualization in Phase 1.

10.2 Affected Files and Lines

- New: `src/preprocessing/deconvolution_sensitivity.R`.
- Output: `data/results/deconvolution_sensitivity/`.

10.3 Implementation Sketch

The sensitivity analysis runs three deconvolution methods on the same input, with the same TMS reference, and reports cross-method agreement. If the per-sample cell-type proportions agree across methods (Spearman $\rho > 0.9$ for each compartment), the strain mismatch concern is empirically defused. If they disagree, the residualization is method-sensitive and the manuscript must report which method was used and why.

```

1  # src/preprocessing/deconvolution_sensitivity.R
2  suppressPackageStartupMessages({
3    library(MuSiC)
4    library(Biobase)
5    library(SummarizedExperiment)
6  })
7
8  run_music <- function(bulk_eset, sc_eset, cell_types) {
9    res <- music_prop(bulk.mtx = exprs(bulk_eset),
10                     sc.eset = sc_eset,
11                     clusters = "cluster",
12                     samples = "sample_id",
13                     select.ct = cell_types)
14    return(res$Est.prop.weighted)
15  }
16
17  run_bMIND <- function(bulk_mat, sc_ref, cell_types) {
18    # Sketch only: requires bMIND package and properly formatted reference
19    library(MIND)
20    res <- bMIND(bulk = bulk_mat,
21               profile = sc_ref,
22               ncore = 4)
23    return(res$frac)
24  }
25
26  run_cibersortx <- function(bulk_mat, sc_signature) {
27    # CIBERSORTx is API-based; this function should call the CIBERSORTx
28    # web service or the Docker container. Returns a matrix of fractions.
29    stop("Implement CIBERSORTx call to local Docker or API.")

```

```

30 }
31
32 # Cross-method comparison
33 prop_music <- run_music(bulk_eset, tms_kidney_sc, cell_types)
34 prop_bmind <- run_bMIND(bulk_mat, tms_signature, cell_types)
35 # prop_cibersortx <- run_cibersortx(bulk_mat, tms_signature)
36
37 # Per-compartment Spearman correlation across methods
38 for (ct in cell_types) {
39   rho_mb <- cor(prop_music[, ct], prop_bmind[, ct], method = "spearman")
40   cat(sprintf("%s: MuSiC vs bMIND Spearman = %.3f\n", ct, rho_mb))
41 }
42
43 # Save side-by-side proportions for downstream sensitivity testing
44 write.table(prop_music,
45             "data/results/deconvolution_sensitivity/music_props.tsv",
46             sep = "\t", quote = FALSE)
47 write.table(prop_bmind,
48             "data/results/deconvolution_sensitivity/bmind_props.tsv",
49             sep = "\t", quote = FALSE)

```

Listing 11: Deconvolution sensitivity analysis.

10.4 Downstream Sensitivity Validation

In addition to comparing the proportions themselves, the residualization in Phase 1 should be re-run using each method's CLR-transformed proportions, and the per-gene rewiring scores in Phase 3 should be computed under each. If the top-decile gene lists agree (Jaccard ≥ 0.7) across methods, the deconvolution choice is not driving the headline biology. If they disagree, the manuscript must report the dependence and the choice of MuSiC must be justified or the sensitivity range reported as a confidence band.

10.5 Effort and Dependencies

Effort: 1 person-day for the sensitivity analysis itself; an additional 1–2 days if the downstream rewiring re-run is required and disagreement is found. **Dependencies:** bMIND and CIBERSORTx must be installable. CIBERSORTx requires a local Docker container or web-API access (free for academic use).

11 Fix 9 — Permutation-Statistic Renaming and Appendix-Tier Full-Pipeline Permutation

11.1 Problem

The Phase 3 observed statistic is row-wise cosine distance between Procrustes-aligned node2vec embeddings. The Phase 6 permutation null is the per-gene sum of absolute Δz over edges incident on that gene (`node_rewiring_abs`), *not* the cosine-distance statistic. These two quantities are correlated under the alternative hypothesis but are not identical, and there is no theoretical guarantee that the relative ranking of the cosine-distance scores matches the ranking of the edge-sum permutation p -values.

11.2 Affected Files and Lines

- `src/statistics/permutation_bootstrap.py` — docstrings, output column names, and `RESULTS.md` terminology.

- Optional: a new module `src/statistics/full_pipeline_permutation.py` for the appendix-tier expensive variant.

11.3 Cheap Fix: Rename and Document

```

1  # In permutation_bootstrap.py docstring, add:
2  """
3  NOTE: This module computes a permutation null for the EDGE-REWIRING statistic
4  (per-gene sum of absolute Delta-z over incident edges), NOT for the cosine-
5  distance EMBEDDING-REWIRING statistic computed in Phase 3. The two statistics
6  are correlated but not identical; the p-values reported here test the
7  hypothesis 'no change in the LIONESS-edge-weight neighborhood of this gene'
8  rather than 'no change in the embedding geometry of this gene.' Users who
9  need a permutation null for the cosine-distance statistic itself should use
10 src/statistics/full_pipeline_permutation.py.
11 """
12
13 # Output column rename (in the per-gene perm_pvals tables):
14 #   p_perm          -> p_perm_edge_rewiring
15 #   q_BH            -> q_BH_edge_rewiring
16 #   rewiring_abs_obs -> edge_rewiring_abs_obs

```

Listing 12: Renaming and documentation update.

11.4 Expensive Fix: Full-Pipeline Permutation on Top Hits

For the top 24 ISS-T-Young hits (or whatever the post-hierarchical-FDR list is), permute the FLT/GC labels within Age×Arm strata $K = 200$ times, and for each permutation re-run Phase 2 (skeleton, LIONESS, edge regression), Phase 3 (node2vec embedding, Procrustes alignment, cosine distance), and record the per-gene cosine-distance statistic. The resulting empirical p -value is a true cosine-distance permutation p -value.

```

1  # src/statistics/full_pipeline_permutation.py
2  def full_pipeline_perm_for_genes(top_genes, n_perms=200,
3                                  n_workers=100, seed=0):
4      """Verify cosine-distance significance for a small list of top genes."""
5      rng = np.random.default_rng(seed)
6      null_distribution = {gene: [] for gene in top_genes}
7
8      for k in range(n_perms):
9          perm_labels = stratified_permute_labels(rng)
10         meta_perm = meta_with_permuted_labels(perm_labels)
11
12         # Re-run Phase 2 with the permuted labels
13         skeleton_k = build_skeleton_with_meta(meta_perm)
14         lioness_k = run_lioness_with_meta(meta_perm)
15         predicted_k = run_edge_regression(meta_perm)
16
17         # Re-run Phase 3 on the permuted predicted networks
18         embeddings_k = run_node2vec_multi_seed(predicted_k)
19         rewiring_k = procrustes_cosine_distance(embeddings_k)
20
21         for gene in top_genes:
22             null_distribution[gene].append(rewiring_k.loc[gene])
23
24         # Empirical p-value per gene
25         p_values = {}
26         for gene in top_genes:
27             obs = observed_rewiring[gene]
28             null_array = np.array(null_distribution[gene])
29             p_values[gene] = (np.sum(null_array >= obs) + 1) / (n_perms + 1)

```



```
30 return p_values
```

Listing 13: Sketch: full-pipeline permutation for top-hit verification.

At $K = 200$ for 24 genes, the run takes roughly 200 full Phase 2/3 iterations. On a 100-CPU cluster with one Phase 2/3 iteration per CPU, this is ~ 200 minutes total; on a single 16-core workstation, ~ 24 hours. Both are tractable.

11.5 Validation

After the cheap rename, all references in `RESULTS.md` and `METHODOLOGY.md` should consistently distinguish “edge-rewiring permutation” from “embedding-rewiring permutation.” After the expensive fix, the cosine-distance permutation p -values for the top hits should be reported alongside the edge-rewiring permutation p -values, and any discrepancy between the two rankings should be flagged.

11.6 Effort and Dependencies

Effort: 1 person-day for the rename; 1 person-day for the expensive variant plus 24 hours of cluster time. **Dependencies:** the existing Phase 2 and Phase 3 modules must be callable from Python (already true).

Tier 2 — Phase 8 Design Interventions

The interventions in this tier do not address the inferential layer; they target the structural cause of the Phase 8 negative result, namely the noise floor of leave-one-out per-sample LIONESS estimation at $n=5$ biological replicates per Age $\times$ Arm $\times$ EnvGroup cell. Each intervention has the potential to move Phase 8 from “network features lose to expression baseline” to “network features add classification value above expression baseline,” but each does so by changing an aspect of the experimental or analytical design rather than fixing a defect in the existing pipeline.

12 Fix 10 — LIONESS Pooling Across Age $\times$ Arm $\times$ EnvGroup Cells

12.1 Problem

Within each Age $\times$ Arm $\times$ EnvGroup cell ($n=5$ mice), the leave-one-out Pearson correlation underlying LIONESS is computed on $N-1=4$ samples. The variance of a 4-sample Pearson correlation is dominated by sampling noise; the LIONESS formula amplifies this noise into the per-sample edge weight. Even after the safety guards (clip $|r| < 0.9995$, cap $|z_s| < 20$), the per-sample LIONESS feature is dominated by the LOO noise rather than by the FLT-vs-GC signal at this sample size.

12.2 Pooling Strategy

Rebuild the LIONESS pool at a coarser stratification level. Three options, in order of aggression:

1. **Pool within Arm** ($N = 20$). Pools FLT, GC, VIV, BSL within an arm; sacrifices Age and EnvGroup specificity. The LOO is now over 19 samples instead of 4.
2. **Pool within Age** ($N = 40$). Pools both arms within an age; sacrifices Arm specificity.
3. **Pool globally** ($N = 80$). Pools everything; sacrifices all design specificity. This is closest to the “standard” LIONESS use case in the literature.

The aggressiveness trades per-sample variance reduction against the risk of confounding the FLT-vs-GC signal with Age- or Arm-specific effects. Option 1 (pool within Arm) is the recommended starting point because it preserves the Arm dimension that the audit identified as the strongest signal carrier and quadruples the LOO sample size simultaneously.

12.3 Affected Files and Lines

- `src/networks/lioness.py` — add a `--pool-strata` argument that accepts `age_arm_envgroup` (current default), `arm`, `age`, or `global`.
- `src/validation/cross_validation.py` — propagate the same flag through the fold-wise rebuild.

12.4 Implementation Sketch

```

1 # In lioness.py main:
2 ap.add_argument("--pool-strata", default="age_arm_envgroup",
3                 choices=["age_arm_envgroup", "arm", "age", "global"],
4                 help="Stratification level for LIONESS pooling. Coarser
5                     pools "
6                     "reduce per-sample LOO variance at the cost of "
7                     "stratum-specificity.")

```

```

7
8 # Pooling logic
9 def get_pool_indices(meta, sample_id, pool_level):
10     """Return indices of samples in the same pool as sample_id."""
11     s = meta.loc[sample_id]
12     if pool_level == "age_arm_envgroup":
13         mask = (meta["Age"] == s["Age"]) & \
14             (meta["Arm"] == s["Arm"]) & \
15             (meta["EnvGroup"] == s["EnvGroup"])
16     elif pool_level == "arm":
17         mask = (meta["Arm"] == s["Arm"])
18     elif pool_level == "age":
19         mask = (meta["Age"] == s["Age"])
20     else: # global
21         mask = pd.Series(True, index=meta.index)
22     return meta.index[mask].tolist()
23
24
25 # In the main LIONESS loop:
26 for s_idx, sample in enumerate(samples):
27     pool = get_pool_indices(meta, sample, args.pool_strata)
28     pool_idx = [i for i, sn in enumerate(samples) if sn in pool]
29     N_pool = len(pool_idx)
30     # Re-derive Sx, Sxx, Sxy on the pool only
31     X_pool = X[:, pool_idx]
32     Sx_pool = X_pool.sum(axis=1)
33     Sxx_pool = (X_pool**2).sum(axis=1)
34     # ... compute LIONESS as before but with N=N_pool

```

Listing 14: Stratified pooling parameter for LIONESS.

12.5 Validation

Three checks:

1. **Per-sample variance.** Bootstrap the LIONESS weights for a representative sample under each pool level and report the median per-edge variance. Pool level **arm** ($N = 20$) should reduce the median per-edge LIONESS standard deviation by approximately a factor of $\sqrt{20/5} = 2$ relative to the current default.
2. **Phase 8b re-run.** Rerun Phase 8b under each pool level and report the network-vs-expression-baseline accuracy table. If LIONESS noise is the binding constraint, accuracy should rise monotonically with pool size.
3. **Confound check.** For each pool level, compute the correlation between the per-sample LIONESS first principal component and the Age (or Arm) covariate. If the correlation is high, the pooling has confounded the FLT-vs-GC signal with Age (or Arm) and the result is uninterpretable.

12.6 Effort and Dependencies

Effort: 1 person-day. **Dependencies:** none.

13 Fix 11 — Alternative Sample-Specific-Network Methods

13.1 Problem

LIONESS is the default for sample-specific networks but is not the only available method. SSN (Liu et al., *Nucleic Acids Research* 2016), CSN (Cell-Specific Networks, Dai et al., 2019), and

scLink are alternatives with different bias-variance tradeoffs at small N . If LIONESS noise is the structural cause of the Phase 8 negative result, an alternative method may have a more favorable noise floor at $n=5$ per cell. If *no* method works at this sample size, the conclusion is that the per-sample classification problem is genuinely intractable at the current cohort size, and Fix 14 (multi-study cohort pooling) becomes the only path forward.

13.2 Affected Files and Lines

- New: `src/networks/alternative_methods.py`.
- New: `src/networks/method_benchmark.py` for cross-method comparison.

13.3 Methods to Implement

- **SSN**. Computes a per-sample z -score against an aggregate reference network: $z_{ij}^{(s)} = (r_{ij}^{(\text{ref}+s)} - r_{ij}^{(\text{ref})}) / \sigma_{ij}$. The variance term σ_{ij} is estimated from the reference network only.
- **CSN**. Tests, for each sample, whether the joint distribution of expression in a small neighborhood around the sample is consistent with independence between gene pairs. Returns a binary edge mask per sample.
- **scLink**. Sparse precision-matrix estimation per sample using a regularized graphical lasso with shrinkage toward a population precision matrix.

13.4 Implementation Sketch

```

1  def ssn_per_sample(X, ref_idx, sample_idx, edge_list):
2      """
3      Sample-Specific Network (Liu 2016):
4          z_e = (r_{ref+s,e} - r_{ref,e}) / sigma_e
5      where sigma_e is computed from the reference network's bootstrap.
6      """
7      ref_X = X[:, ref_idx]
8      n_ref = len(ref_idx)
9
10     # Reference correlations
11     r_ref = pearson_edges(ref_X, edge_list)
12
13     # Reference bootstrap variance
14     n_boot = 500
15     boot_r = np.empty((n_boot, len(edge_list)))
16     rng = np.random.default_rng(0)
17     for b in range(n_boot):
18         idx = rng.choice(ref_idx, size=n_ref, replace=True)
19         boot_r[b] = pearson_edges(X[:, idx], edge_list)
20     sigma_e = boot_r.std(axis=0)
21
22     # ref + sample correlation
23     aug_idx = list(ref_idx) + [sample_idx]
24     r_aug = pearson_edges(X[:, aug_idx], edge_list)
25
26     z = (r_aug - r_ref) / np.maximum(sigma_e, 1e-6)
27     return z
28
29
30  def csn_per_sample(X, sample_idx, k_neighbors=5):
31      """
32      Cell-Specific Network (Dai 2019, simplified):
33      For each gene pair (i,j), test whether sample s is consistent with
34      independence in a local neighborhood around s.

```

```

35     """
36     # Identify k nearest neighbors of sample s in expression space
37     # Compute per-gene-pair contingency table on this neighborhood
38     # Return chi-square test statistic per edge
39     raise NotImplementedError("Implement per the Dai 2019 algorithm.")

```

Listing 15: Wrapper for SSN.

13.5 Validation

Run each method through Phase 8b under leakage-safe CV and report the network-vs-expression-baseline table side by side with LIONESS. The method with the largest network advantage (or, equivalently, the smallest negative network advantage) is the new default. If all methods produce negative network advantage at OSD-771’s sample size, document that the per-sample-network classification problem is intractable at $n=5$ per cell regardless of method choice.

13.6 Effort and Dependencies

Effort: 3 person-days for the three method wrappers; an additional 1 day for the benchmark.

Dependencies: scLink requires R; SSN and CSN are pure Python.

14 Fix 12 — Alternative Feature Engineering for Phase 8

14.1 Problem

The current Phase 8 features are “top 50 node-strength features by training-fold variance + 10 PCA components.” Node strength (sum of $|z|$ over incident edges) is a coarse summary; it integrates over edges that may carry signal in opposite directions, producing a feature that is constant across many samples even when the underlying network differs. Per-edge LIONESS values, pairwise embedding distances, or graph-theoretic centrality measures may carry signal that node-strength averages out.

14.2 Affected Files and Lines

- `src/validation/cross_validation.py` — the feature-engineering function.

14.3 Feature Sets to Compare

1. **Current.** Top-50 node-strength features by training variance + 10 PCA components.
2. **Per-edge.** All $\sim 180,000$ LIONESS edge weights, with L_1 -regularized logistic regression to enforce sparsity. The classifier picks its own discriminative edges.
3. **Embedding distance.** Pairwise cosine distance between each test sample’s per-mouse node2vec embedding and the per-condition consensus embedding. The feature for sample s is the cosine distance from FLT consensus and from GC consensus; classification reduces to “which consensus is the test sample closer to.”
4. **Graph centrality.** Betweenness centrality, eigenvector centrality, and clustering coefficient computed on the per-sample LIONESS network thresholded at the top- k edges.

14.4 Implementation Sketch

```

1  from sklearn.linear_model import LogisticRegressionCV
2  from sklearn.preprocessing import StandardScaler
3

```

```

4 def per_edge_features(z_lioness):
5     """Treat every edge as its own feature."""
6     return z_lioness # shape: (n_samples, n_edges)
7
8
9 def evaluate_per_edge_l1(X_train, y_train, X_test, y_test):
10    scaler = StandardScaler().fit(X_train)
11    Xs_train = scaler.transform(X_train)
12    Xs_test = scaler.transform(X_test)
13    clf = LogisticRegressionCV(
14        Cs=10, penalty='l1', solver='liblinear',
15        cv=4, scoring='accuracy', max_iter=2000
16    ).fit(Xs_train, y_train)
17    return clf.score(Xs_test, y_test), clf.predict_proba(Xs_test)

```

Listing 16: Per-edge feature set with L1 logistic regression.

```

1 def embedding_distance_features(test_emb, flt_consensus, gc_consensus):
2     """For each test sample, return [d(test, flt), d(test, gc), d_diff]."""
3     from scipy.spatial.distance import cosine
4     feats = []
5     for s in range(test_emb.shape[0]):
6         d_flt = cosine(test_emb[s], flt_consensus)
7         d_gc = cosine(test_emb[s], gc_consensus)
8         feats.append([d_flt, d_gc, d_flt - d_gc])
9     return np.array(feats)

```

Listing 17: Embedding-distance feature.

14.5 Validation

Run all four feature sets through Phase 8b under leakage-safe CV and report the resulting network-vs-expression-baseline table. The feature set with the largest positive network advantage in any stratum is reported. If none produces positive network advantage, the conclusion is that feature engineering is not the binding constraint and Fix 10 (LIONESS pooling) or Fix 14 (cohort pooling) is required.

14.6 Effort and Dependencies

Effort: 1.5 person-days. **Dependencies:** the per-edge feature set requires care with leakage-safe scaling; the embedding-distance feature requires per-fold rebuild of the FLT and GC consensus embeddings.

15 Fix 13 — Continuous Classification Targets

15.1 Problem

The current Phase 8 target is binary FLT-vs-GC. At $n=5$ per stratum the binary task discards most of the available variation: a continuous biological response is being thresholded into a 0/1 label, and the classifier must squeeze 40 samples into a binary decision. A continuous target derived from independent kidney-injury biomarkers (KIM-1, NGAL, HAVCR1) lets the classifier exploit ordinal information.

15.2 Affected Files and Lines

- New: `src/validation/continuous.target.py`.

- Required input: kidney-injury biomarker expression for each sample (already present in the OSD-771 counts matrix; specific genes need to be looked up by symbol).

15.3 Stress-Score Definition

The stress score for sample s is the residualized z -score of:

- *Havcr1* (Kim-1) — canonical proximal-tubule injury marker.
- *Lcn2* (NGAL) — distal-tubule injury marker.
- *Trf* — iron-homeostasis stress marker (already among the top hits).
- *Spp1*, *Vim*, *Cdkn1a* — additional kidney-stress markers.

The score is the first principal component of the residualized z -scores, scaled to $[0, 1]$.

15.4 Implementation Sketch

```

1  def kidney_stress_score(rtech, marker_genes):
2      """Construct a continuous stress score from kidney injury markers."""
3      from sklearn.decomposition import PCA
4      from sklearn.preprocessing import StandardScaler
5
6      Y = rtech.loc[marker_genes].T # (n_samples, n_markers)
7      Y_scaled = StandardScaler().fit_transform(Y)
8      pc1 = PCA(n_components=1).fit_transform(Y_scaled).ravel()
9      # Min-max scale to [0, 1] for interpretability
10     return (pc1 - pc1.min()) / (pc1.max() - pc1.min())
11
12
13  def regression_phase8(X_train, y_train, X_test, y_test):
14      """Continuous-target version of Phase 8, using Spearman rho as metric."""
15      from sklearn.ensemble import RandomForestRegressor
16      from scipy.stats import spearmanr
17      rf = RandomForestRegressor(n_estimators=100, max_depth=5, random_state=0)
18      rf.fit(X_train, y_train)
19      y_pred = rf.predict(X_test)
20      rho, p = spearmanr(y_pred, y_test)
21      return rho, p

```

Listing 18: Continuous stress-score regression target.

15.5 Validation

The Spearman correlation between predicted and observed stress scores should be reported per fold and per stratum, alongside the existing classification accuracy. The new metric to beat is “network features predict stress score with $\rho > \rho_{\text{expression}}$.” If the network features predict stress scores significantly better than expression alone (one-sided permutation $p < 0.05$), this is positive evidence for the network framework even if the binary classifier remains negative.

15.6 Effort and Dependencies

Effort: 2 person-days. **Dependencies:** the marker-gene list must be intersected with the 2500-gene panel; if any are missing, the panel must be expanded or the score must be recomputed on the full residualized matrix.

16 Fix 14 — Multi-Study Cohort Pooling for LAR-Young (OSD-102 + OSD-771)

16.1 Problem

The binding constraint on per-sample classification is $n=5$ per Age $\times$ Arm $\times$ EnvGroup cell. At this sample size no analytical fix produces a stable per-sample LIONESS feature set, because leave-one-out correlations are computed on $N-1=4$ samples and the resulting per-mouse weights inherit excessive sampling variance. The only intervention that genuinely changes the noise floor is to add more biological replicates.

The originally proposed pooling partner (OSD-568, RRRM-1 kidney) has been removed: it uses a different mouse strain (BALB/cAnNTac vs C57BL/6NTac) and likely a different assay (miRNA-seq or scRNA-seq, per the OSDR release announcement). The strain mismatch is fatal — ComBat-seq cannot remove BALB/c-vs-C57BL/6 baseline biology without also removing real spaceflight signal — and the assay mismatch makes count-level pooling impossible.

The viable pooling partner is **OSD-102 (RR-1 NASA Validation Flight)**: female C57BL/6J kidney, $n=6$ FLT + $n=6$ GC, 16 weeks at sacrifice, 37-day flight, bulk RNA-seq, RNAlater $\rightarrow -80^\circ\text{C}$, NuRFB diet, Rodent Habitat hardware, 12h/12h light cycle. Only the C57BL/6J vs C57BL/6NTac sub-strain difference remains, which is small enough (1–2% of kidney genes show baseline divergence) to be handled as a study-as-batch covariate in ComBat-seq.

The pool is stratum-specific, not global. OSD-102 has no ISS-T arm, no aged cohort, and no basal control, so pooling can only augment one of the 16 RRRM-2 cells: *LAR-Young*. After pooling, the LAR-Young cell has $n=11$ FLT + $n=11$ GC instead of $n=5+5$, which means LIONESS LOO correlations are computed on 21 samples instead of 4 — a per-sample variance reduction of $\sqrt{21/4} \approx 2.3$. The other three RRRM-2 cells (ISS-T-Young, ISS-T-Old, LAR-Old) cannot be augmented this way and continue at $n=5$. Fix 14 is therefore a targeted intervention on the LAR-Young classification problem, not a global remedy. If LIONESS noise at $n=5$ is the binding constraint on Phase 8 (the diagnosis converged on in the audit), this is the largest noise-floor reduction available without running new mice and is plausibly large enough to flip the LAR-Young classifier from below-chance to above-expression-baseline.

16.2 Affected Files and Lines

- New: `src/validation/multi_study_pool.py`.
- Required preprocessing: cross-study batch correction.
- New: `src/preprocessing/cross_study_harmonization.R`.

16.3 Implementation Strategy

The pooled analysis must address inter-study technical variation while preserving the FLT-vs-GC biology. Four preprocessing steps:

1. **Restrict RRRM-2 to LAR-Young.** Keep only the 10 mice ($n=5$ FLT + $n=5$ GC; pooling 5 HGC and 5 VIV with 6 OSD-102 GC) from the LAR-Young cell. Optionally also include LAR-Young VIV as additional GC ($n=5$); decide pre-registration, not post-hoc.
2. **Restrict OSD-102 to individual mice.** Use only M23–M28 (FLT) and M33–M38 (GC), $n=12$ total. Exclude all four pool samples (“Pool of all FLT samples,” “Pool of all GC samples,” “Pool of all FLT and GC samples,” “Empty Pool”).

3. **Gene-name harmonization.** Map both studies to a common gene-ID space (Ensembl IDs are the most reliable; both NASA GeneLab releases include Ensembl). Drop genes not present in both.
 4. **ComBat-seq batch correction.** Apply ComBat-seq (Zhang et al., 2020) with study as the batch and FLT-vs-GC as the biology to preserve. Critically, do *not* also try to preserve sex (both studies female), age (both 16 weeks at sacrifice), or strain (study-as-batch already absorbs the C57BL/6J vs C57BL/6NTac sub-strain difference). Adding a redundant covariate to `covar_mod` can over-correct.
 5. **QC harmonization.** Apply a unified CPM and gene-detection filter to the combined cohort.
- After harmonization, the pipeline runs as before but with $n_{\text{LAR-Young}}=11$ FLT + $n=11$ GC (or 11+16 if VIV is included as additional GC).

16.4 Implementation Sketch

```

1 # src/preprocessing/cross_study_harmonization.R
2 suppressPackageStartupMessages({
3   library(sva)           # for ComBat_seq
4   library(edgeR)
5 })
6
7 harmonize_lar_young <- function(counts_771, meta_771, counts_102, meta_102,
8                                include_viv_as_gc = FALSE) {
9   # Restrict OSD-771 to LAR-Young
10  is_lar_young <- meta_771$Age == "Young" & meta_771$Arm == "LAR"
11  if (include_viv_as_gc) {
12    keep_771 <- is_lar_young &
13      meta_771$EnvGroup %in% c("FLT", "HGC", "VIV")
14  } else {
15    keep_771 <- is_lar_young & meta_771$EnvGroup %in% c("FLT", "HGC")
16  }
17  meta_771_sub <- meta_771[keep_771, ]
18  c771_sub <- counts_771[, keep_771]
19
20  # Restrict OSD-102 to individual mice (drop pool samples)
21  pool_samples <- grepl("^Pool|^Empty Pool", meta_102$'Source Name')
22  meta_102_sub <- meta_102[!pool_samples, ]
23  c102_sub <- counts_102[, !pool_samples]
24
25  # Common Ensembl IDs
26  common <- intersect(rownames(c771_sub), rownames(c102_sub))
27  combined <- cbind(c771_sub[common, ], c102_sub[common, ])
28
29  # Build a unified meta with FLT-vs-GC label and study-as-batch
30  treatment <- c(
31    ifelse(meta_771_sub$EnvGroup == "FLT", "FLT", "GC"),
32    ifelse(grepl("FLT", meta_102_sub$'Factor Value: Spaceflight'), "FLT",
33      "GC")
34  )
35  study <- c(rep("OSD-771", nrow(meta_771_sub)),
36    rep("OSD-102", nrow(meta_102_sub)))
37  meta_combined <- data.frame(sample_id = colnames(combined),
38    study = study,
39    treatment = treatment)
40
41  # ComBat-seq: study as batch, treatment as the only biology to preserve.
42  # DO NOT add sex/age/strain to covar_mod -- they are confounded with study
43  # at this restricted slice (both studies female, both 16 weeks).
44  biology <- model.matrix(~ treatment, data = meta_combined)
45  corrected <- ComBat_seq(

```

```

45     counts      = as.matrix(combined),
46     batch       = meta_combined$study,
47     group       = NULL,
48     covar_mod    = biology
49   )
50
51   list(counts = corrected, meta = meta_combined)
52 }

```

Listing 19: LAR-Young cohort harmonization for OSD-771 + OSD-102 pooling.

16.5 Validation

Three checks, plus a fallback path:

1. **Batch effect resolution.** PCA of the post-ComBat-seq matrix on the LAR-Young pool should not separate by study (OSD-771 vs OSD-102). If it does, ComBat-seq has under-corrected — usually a sign that the residual sub-strain difference is too strong to absorb with study-as-batch alone.
2. **Biology preservation.** PCA of the post-ComBat-seq matrix should still separate by treatment (FLT vs GC). If it doesn't, ComBat-seq has over-corrected.
3. **Phase 8b re-run.** Phase 8b on the pooled LAR-Young cohort should show a substantially smaller negative network advantage relative to the original RRRM-2-only run — ideally crossing zero into positive territory — if the LIONESS-noise hypothesis is correct. If LAR-Young network advantage stays negative even at $n=11$ per condition, the LIONESS-noise hypothesis is partially refuted and the residual negative result is more likely a genuine “network rewiring is redundant with expression” phenomenon (§4 of the audit’s diagnosis discussion).

Fallback if either PCA check fails. If batch effect resolution fails, fall back to a fixed-effects gene-level meta-analysis: compute per-gene effect estimates separately on RRRM-2 LAR-Young and on OSD-102, then combine them with sample-size-weighted Stouffer (or, better, a fixed-effects inverse-variance-weighted average of standardized effects). This is a weaker form of pooling because it operates at the per-gene level rather than the per-mouse network level, and it cannot rebuild a pooled LIONESS network — so it does not address the Phase 8 noise floor. But it does provide a defensible cross-cohort effect estimate that is robust to count-level batch contamination.

16.6 Effort and Dependencies

Effort: 4 person-days. **Dependencies:** OSD-102 must be downloadable from GeneLab; the metadata harmonization between OSD-102 and OSD-771 (different field names: OSD-102 uses ISA-Tab columns like “Factor Value: Spaceflight,” OSD-771 uses RRRM-2 schema with EnvGroup) must be done manually or via an explicit lookup table; the four pool samples in OSD-102 must be excluded.

17 Implementation Roadmap

17.1 Dependency Graph

The fixes have nontrivial dependencies. The recommended ordering, with rationale:

1. **Fix 7 (config drift)** first — 1 hour, no dependencies, prevents downstream confusion about which parameter values are actually used.
2. **Fix 6 (anchor pre-registration)** second — 1 hour, modifies the gene panel marginally, will rerun Phase 3 once.
3. **Fix 1 (Brown’s aggregation)** third — 0.5 days, removes the most numerically broken outputs.
4. **Fix 5 (silent-shifter null)** fourth — 0.5 days, can run in parallel with Fix 1.
5. **Fix 2 (hierarchical FDR)** fifth — 1 day, gates Fix 3 (RESULTS reconciliation, deferred until all docs are regenerated post-fix-batch).
6. **Fix 9 (statistic rename)** sixth — 1 day, can run in parallel with Fix 2.
7. **Fix 3 (RESULTS reconciliation)** — deferred. Per the abstract, all documentation overhauls (README, RESULTS, METHODOLOGY) are batched and regenerated at the end, not interleaved with the inferential fixes. Mark Fix 3 “deferred to post-fix documentation pass.”
8. **Fix 8 (MuSiC sensitivity)** seventh — 1 day, can run in parallel with the inferential fixes.
9. **Fix 4 (external validation)** eighth — 2 days, requires Fix 2 to fix the hypothesis set.
10. **Fix 10 (LIONESS pooling)** ninth — 1 day, first Tier-2 intervention.
11. **Fix 12 (feature engineering)** tenth — 1.5 days, runs alongside Fix 10.
12. **Fix 11 (alternative SSN methods)** eleventh — 3 days.
13. **Fix 13 (continuous targets)** twelfth — 2 days.
14. **Fix 14 (cohort pooling)** thirteenth — 4 days, the most expensive and most decisive.

17.2 Total Effort

Tier 1: ≈ 7 person-days (without external-cohort data ingestion delays). Down from the earlier 8-day estimate because the Phase 8 narrative reframe has been removed — documentation is now batched and regenerated at the end rather than interleaved. Tier 2: ≈ 12 person-days plus cluster time. Total: ≈ 19 person-days for one engineer working full-time on remediation, plus a final 0.5–1 day documentation pass at the end to regenerate `README.md`, `RESULTS.md`, and `METHODOLOGY.md` against the post-fix realized numbers. Roughly 4 calendar weeks at 5 days/week.

17.3 Minimum-Viable Path to Publication

If the goal is to reach a publishable manuscript as quickly as possible, the minimum-viable path is Fixes 7, 1, 2, 5, and 4 (in that order). This is roughly 4 person-days plus the OSD-102 (and optionally OSD-513) download and run. Documentation regeneration (the work that the original Fix 2 “Phase 8 narrative reframe” and Fix 3 “RESULTS.md reconciliation” would have addressed) is deferred to a single batch at the end and is not on this minimum-viable critical path. The result is a manuscript with:

- Statistically defensible gene-level FDR claims (hierarchical Benjamini–Bogomolov on the post-Brown’s-method aggregation).
- Externally validated hypothesis set for the LAR-Young arm (the surviving genes after OSD-102 replication, with OSD-513 as the sex-stratification check).
- Correctly silent-shifter-null-calibrated gene categories.
- A documentation pass (deferred to the very end) that reflects the post-fix realized numbers and the leakage-safe Phase 8 negative result.

The remaining Tier-1 fixes (Fix 6 anchor pre-registration, Fix 8 MuSiC sensitivity, Fix 9 permutation rename, Fix 3 RESULTS reconciliation as part of the deferred doc pass) sharpen the manuscript but do not gate publication. The Tier-2 fixes are appropriate for a follow-up study.

17.4 Risk Register

Each fix carries a small risk that should be tracked:

#	Risk	Mitigation
1	Empirical-Brown over-corrects when edge dependence is weak.	Fall back to label-permutation aggregation if Stouffer and Brown disagree by less than 1%.
2	Hierarchical procedure rejects too few; the manuscript ends up with no gene-level hits.	Report both the hierarchical and the unadjusted gene lists; let reviewers see both.
4	OSD-102 fails to replicate the LAR-Young hits; or OSD-513 reveals all surviving hits are sex-DE.	Plan for both outcomes in advance. The pathway-level findings (translation machinery, PPAR signaling) likely survive even if individual genes do not, and a “female-specific space-flight signature” result is itself publishable. ISS-T-Young hits cannot be validated without a future ISS-T cohort; report as candidates.
6	Pre-registered anchor list intersection is too small.	Expand <code>anchor_genes.yaml</code> from MGI house-keeping and ribosomal lists.
10	Pooling within Arm reintroduces Age confounding.	The validation step explicitly checks for this; if Age confounding is detected, escalate to Pool-within-Age and document.
11	All alternative methods produce the same negative result.	This is the informative outcome — it tells you the per-sample classification problem is intractable at $n=5$ and Fix 14 is required.
14	ComBat-seq under- or over-corrects.	The two PCA validation checks catch both failure modes. If either fails, fall back to a fixed-effects-only meta-analysis at the gene level rather than a pooled per-sample analysis.

18 Acceptance Criteria

The remediation is complete when:

1. `phase6_regression/` no longer produces `p_stouffer = 0.0` for any gene with median per-edge $|t| < 2$.
2. `phase6_uncertainty/` reports both the existing edge-rewiring permutation p -values *and* a hierarchical-Benjamini–Bogomolov-corrected gene list.
3. `phase5_silent_shifters_strict/` reports observed-versus-expected counts under the null per Fix 5.

4. `phase3_rewiring/anchors.txt` matches the union of pre-registered anchors in `config/anchor_genes.yaml` intersected with the gene panel.
5. `config/hyperparameters.yaml` matches `src/` verbatim, verified by `tests/test_config_consistency.py`.
6. `data/results/external_replication/OSD-102/replication_report.tsv` exists and reports the per-gene replication outcomes on OSD-102 for the LAR-Young hypothesis set; `data/results/external_` exists and reports the sex-stratified replication outcomes on OSD-513.
7. Documentation regeneration pass: `README.md`, `RESULTS.md`, and `METHODOLOGY.md` are regenerated as a single batch after all inferential and Tier-2 fixes have landed, against the post-fix realized run. They reference the post-fix run identifier (replacing `run_20260408_191759_2500g`) and frame Phase 8 according to whichever outcome the Tier-2 interventions produce (negative result if they fail, positive result if they succeed).
8. `data/results/deconvolution_sensitivity/` reports cross-method Spearman correlations for each cell-type proportion.
9. Optional (Tier 2): `data/results/phase8.pooled_arm/` reports Phase 8b accuracy under LIONESS pooling within Arm; if positive network advantage emerges, the manuscript adopts this as the new default.

At that point the pipeline's inferential layer is publication-defensible, its narrative is consistent with the realized outputs, and the path to a stronger Phase 8 result has been mapped (even if not all Tier 2 interventions have been attempted).

This guide is a companion to `critique_report.tex`. It translates the audit's identified weaknesses into concrete file-level remediations with code sketches and validation procedures. All Tier 1 fixes (1–9) are necessary for publication; the Tier 2 fixes (10–14) target the structural cause of the Phase 8 negative result and are appropriate for a follow-up study unless the authors have the engineering bandwidth to land them on the current submission timeline. Documentation overhauls (`README`, `RESULTS`, `METHODOLOGY`) are deliberately deferred to a single regeneration pass at the end rather than interleaved with the technical fixes.